Talk transcript: Building quantum codes with transversal CCZ

Virginia Tech Applied Algebra Seminar, October 22, 2026. Joint work with Ken Brown. About 37 minutes, then discussion.

How to use this transcript

Each section is one frame of the deck, in order. The heading gives the displayed slide number. The five unnumbered outline frames between parts have no script. Advance past each one after about ten seconds.

- **Clock** is the planned start and end time. **Budget** is the time for the frame. **Clicks** is the number of overlay steps after the first view.
- **[click]** marks where to advance. The text after it describes what has just appeared.
- *(pause)* means stop for about two seconds and let the slide be read.
- *(ask, then wait 7 seconds)* marks a question to the audience. A fallback answer follows in case nobody answers.
- Spoken text is written at about 135 words per minute. Read it aloud at a steady pace with the clock running, and mark where you run over.
- **Background (not spoken)** boxes on slides 2 to 8 and on a few later slides collect the facts behind each sentence, with short answers to likely questions. Read them before rehearsing, not during.

Terms and notation follow the standard literature and our paper: Pauli strings $X(a)$ and $Z(b)$, stabilizers and stabilizer generators, X-type and Z-type stabilizers, logical operators, the CSS code of $\mathcal{C}_2 \subseteq \mathcal{C}_1$, quasi-transversal, triorthogonal, column indicator. Two terms come from our paper and are defined on the slides: the signed weight and the logical label.

Winston's "How to Speak" applied to this talk

These heuristics are from Patrick Winston's MIT lecture "How to Speak" (MIT OpenCourseWare RES.TLL-005, IAP 2018). Each one is built into the script.

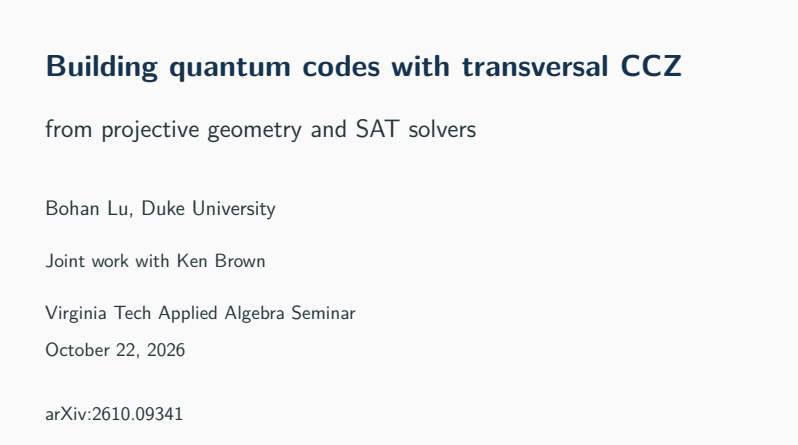
- **Worked examples first.** The $[\![8, 3, 2]\!]$ code is the only worked example before $\mathcal{Q}_{48}$: it carries the CSS background on slide 5, appears in the table of three codes on slide 8, and gives logical CCZ on slides 9 to 11, before the result on slide 14.
- **Cycling.** The three steps of the construction are said at slides 14, 15 and 28: "Geometry fixes the stabilizers S, a search finds the logical rows K, and linear algebra over $\mathbb{Z}_4$ finds the signs."
- **Fences.** Each of these is said once, at the slide where it applies:
  - quasi-transversal CCZ versus a $T/T^\dagger$ pattern with no correction (slides 13 and 27)
  - 8-divisibility is a design choice, not part of the conditions (slide 15)
  - the 47-qubit code of Jacinto et al. is not $\mathcal{Q}_{48}$ with one point removed (slide 27).
- **Verbal punctuation.** The outline frames between parts mark where each part of six begins. Start the next slide with its transition sentence.
- **Ask a question.** Two planned questions, at slides 10 and 18, each with a 7-second wait.
- **Collaborators first.** Ken Brown and Robert Calderbank are credited on the title slide.
- **Ending.** Slide 28 is read as contributions, and the talk ends with a salute to the audience instead of "thank you".

Pacing checkpoints

Reach this point	Clock	If about one minute behind
Start of slide 6	5:45	On slide 5, show the $x = 100$ pair, name the $x = 111$ pair, and go straight to the edge.
Start of slide 12	12:30	Give the timeline in two sentences. Keep the odd-triple point and the parity/sign distinction.
Start of slide 20	22:10	On slide 19, give the formula, the Q48 entry and the census qualification only.
Start of slide 25	30:10	Give the Reed–Muller argument in two sentences. Name the four stages of the bound without detail.
Slide 28	35:40	Read the contributions and two open questions. Stop at 37:10.

Do not save time by dropping the sign-feasibility distinction or the hypotheses of the lower bound. If a question interrupts a derivation, give a short answer and offer to return to it in the discussion.

Title slide

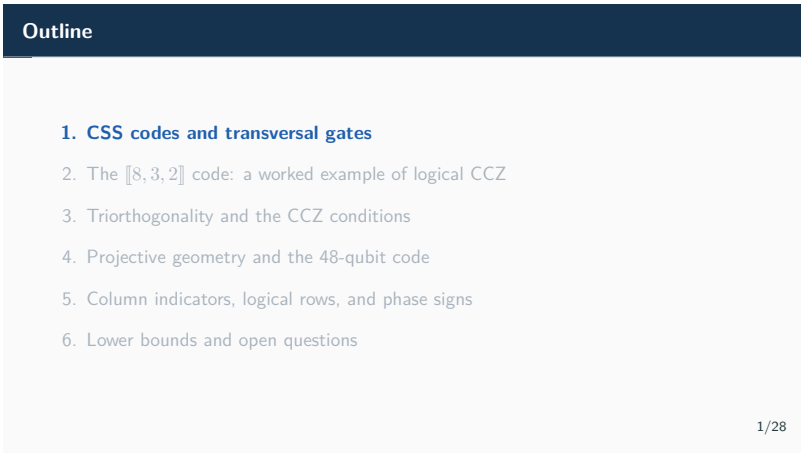


Clock 0:00–0:15 · **Budget** 15 s · **Clicks** 0

On screen: title, Bohan Lu and Ken Brown, arXiv number.

Thank you for the invitation. This is joint work with Ken Brown at Duke, and I also thank Robert Calderbank for pointers on the classical coding theory.

Slide 1. Outline



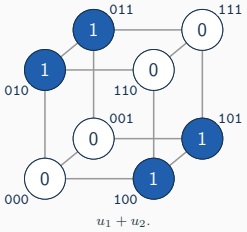
Clock 0:15–1:00 · **Budget** 45 s · **Clicks** 0

On screen: the six parts of the talk, with part 1 highlighted.

The talk has six parts. Part one is CSS codes and transversal gates. Part two works through the $[\![8, 3, 2]\!]$ code as an example of logical CCZ. Part three gives the triorthogonality and CCZ conditions. Parts four and five build a 48-qubit code from projective geometry, then find its logical rows and phase signs. Part six gives the lower bounds and open questions.

Slide 2. Generator rows on the cube

Generator rows on the cube



$u_1 + u_2$.

Table of values: each column is a point $u \in \mathbb{F}_2^3$ (one of $n = 8$ qubits), each row a Boolean function f, and the entry is $f(u)$. So f is a row in $\mathbb{F}_2^8$, of weight $|f| = \#\{u : f(u) = 1\}$.

$f \setminus u$	000	001	010	011	100	101	110	111
u_1	0	0	0	0	1	1	1	1
u_2	0	0	1	1	0	0	1	1
u_1u_2	0	0	0	0	0	0	1	1
$u_1 + u_2$	0	0	1	1	1	1	0	0

Row operations: overlap $c \wedge c'$ (entrywise product), XOR $c \oplus c'$ (entrywise sum), dot product $c \cdot c' = |c \wedge c'| \bmod 2$.

Dictionary: the row of fg is the overlap of the rows f, g. The row of $f + g$ is their XOR.

2/28

Clock 1:00–2:00 · **Budget** 60 s · **Clicks** 4

On screen: the cube $\mathbb{F}_2^3$ with one qubit at each point, and a table with the columns labelled 000 to 111. The clicks add the rows of u_1, u_2, u_1u_2 and $u_1 + u_2$, each also drawn on the cube, with the row operations and the dictionary between functions and rows.

In the table of values, each column is a point u of $\mathbb{F}_2^3$, one qubit, and each row is a Boolean function f, with entry $f(u)$. So f is a row of length eight, and its weight $|f|$ is the number of ones.

[click] u_1 gives the row 00001111, the face where the first coordinate is one.

[click] u_2 gives the row 00110011.

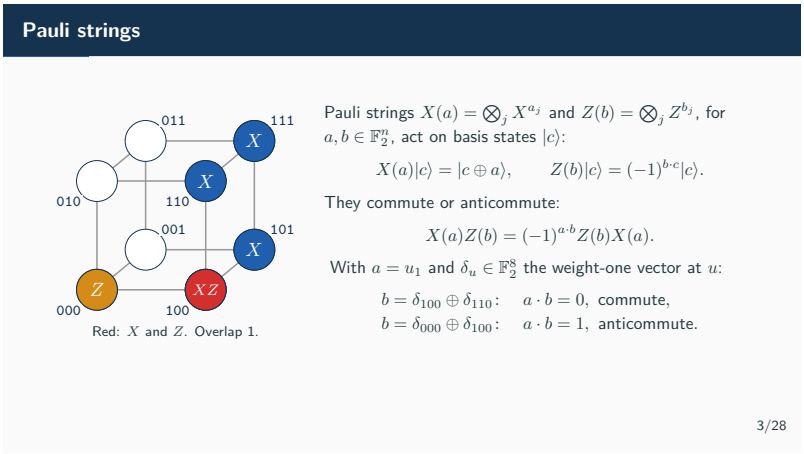
[click] Three operations on rows c and c': the overlap $c \wedge c'$ is the entrywise product, the XOR $c \oplus c'$ is the entrywise sum, and the dot product $c \cdot c'$ is the parity of the overlap. Here is the dictionary we use later. The row of a product fg is the overlap of the rows of f and g, so u_1u_2 gives 00000011, an edge of weight two.

[click] The row of a sum $f + g$ is the XOR of their rows, so $u_1 + u_2$ gives 00111100.

Background (not spoken)

- **Rows and functions.** Functions on $\mathbb{F}_2^3$ are built with sums and products. Rows in $\mathbb{F}_2^8$ are combined with overlap, XOR and dot product. Evaluation turns a product of functions into the overlap of their rows and a sum into the XOR. Never write $u_1 \wedge u_2$ or $u_1 \oplus u_2$: the u_i are functions. The dictionary is used on slide 3 (XOR and dot product), slide 10 (overlaps of rows) and slide 21 (an overlap is a monomial).
- **Dot product versus overlap.** $c \cdot c'$ is a single bit, $\sum_i c_i c'_i \bmod 2$. The overlap $c \wedge c'$ is a vector. They are related by $c \cdot c' = |c \wedge c'| \bmod 2$.
- **Why the cube.** The cube is only a picture of $\mathbb{F}_2^3$. Faces are the sets $u_i = 1$, two faces meet in an edge, and three faces meet in a vertex. The 48-qubit code later uses the same idea on 48 points of $\mathbb{F}_2^6$.

Slide 3. Pauli strings



Clock 2:00–3:00 · **Budget** 60 s · **Clicks** 3

On screen: the Pauli strings $X(a)$ and $Z(b)$ and their action on $|c\rangle$, with $X(u_1)$ on the cube. The clicks add the commutation rule and two examples.

We use binary vectors a and b to label the positions where we place Pauli X and Z operators in the Pauli strings $X(a)$ and $Z(b)$. On the left is $X(u_1)$, an X on each point of the face u_1. On a basis state, $X(a)$ sends $|c\rangle$ to $|c \oplus a\rangle$, and $Z(b)$ multiplies $|c\rangle$ by $(-1)^{b\cdot c}$.

[click] Pauli strings commute or anticommute: $X(a)Z(b) = (-1)^{a\cdot b} Z(b)X(a)$. Each qubit in the overlap contributes one minus sign.

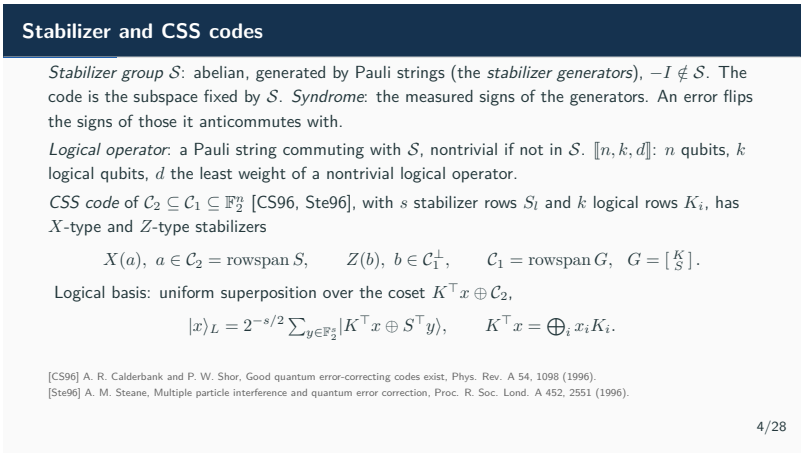
[click] Take $a = u_1$. Put Z on the edge from 100 to 110, so b has ones at those two points: $b = \delta_{100} \oplus \delta_{110}$, where δ_u is the weight-one vector at u. Both points are in the face, shown in red, so $a \cdot b = 0$ and the strings commute.

[click] Now put Z on the edge from 000 to 100, so $b = \delta_{000} \oplus \delta_{100}$. Only 100 is in the face, so $a \cdot b = 1$ and they anticommute.

Background (not spoken)

- **Notation.** $X(a)$ and $Z(b)$ with $a, b \in \mathbb{F}_2^n$ is the notation of Camps-Moreno et al. and of Rengaswamy, Calderbank and coauthors. The Pauli group is generated by $\pm X(a)Z(b)$, with $Y = iXZ$ on a single qubit. For general strings, $X(a)Z(b)$ and $X(a')Z(b')$ commute up to the sign $(-1)^{a\cdot b' + b\cdot a'}$, the symplectic inner product. The talk only needs the case $X(a)$ against $Z(b)$.
- **Why Pauli strings suffice.** I, X, Z and XZ span the 2×2 complex matrices. The Knill–Laflamme conditions are linear in the error, so a code that corrects a set of Pauli errors also corrects every linear combination of them.

Slide 4. Stabilizer and CSS codes



Clock 3:00–4:15 · **Budget** 75 s · **Clicks** 2

On screen: stabilizer group, stabilizer code, syndrome, logical operator and $[\![n, k, d]\!]$. The clicks add the CSS code of $\mathcal{C}_2 \subseteq \mathcal{C}_1$ with S, K and G, and the logical basis $|x\rangle_L$.

Commuting Pauli strings define a code. A stabilizer group $\mathcal{S}$ is an abelian group generated by Pauli strings, the stabilizer generators, that does not contain $-I$. The code is the subspace fixed by $\mathcal{S}$. The syndrome is the measured signs of the generators, and an error flips the signs of those it anticommutes with. A logical operator is a Pauli string that commutes with $\mathcal{S}$. It is nontrivial if it is not in $\mathcal{S}$. An $[\![n, k, d]\!]$ code has n qubits and k logical qubits, and d is the least weight of a nontrivial logical operator.

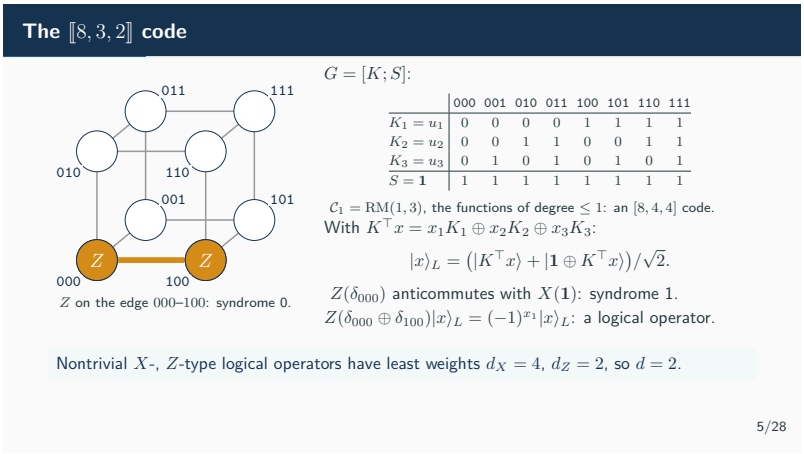
[click] A CSS code comes from two nested binary codes, $\mathcal{C}_2 \subseteq \mathcal{C}_1$. The X-type stabilizers are $X(a)$ for $a \in \mathcal{C}_2$, and the Z-type stabilizers are $Z(b)$ for $b \in \mathcal{C}_1^\perp$. $\mathcal{C}_2$ is spanned by the s stabilizer rows of S, and $\mathcal{C}_1$ by G, the k logical rows of K stacked on S.

[click] The logical basis state $|x\rangle_L$ is the uniform superposition over the coset $K^\top x \oplus \mathcal{C}_2$. Here $K^\top x$ is the sum of the logical rows picked by x. The next slide fills in each symbol for the $[\![8, 3, 2]\!]$ code.

Background (not spoken)

- **Why an anticommuting error flips the eigenvalue.** If $P|\psi\rangle = |\psi\rangle$ and $PE = -EP$, then $P(E|\psi\rangle) = -E|\psi\rangle$. So the corrupted state is in the -1 eigenspace of P.
- **Why eigenvalues are ± 1.** Each stabilizer is a Hermitian Pauli string with $P^2 = I$. The group must not contain $-I$.
- **Weight.** The weight of $X(a)Z(b)$ is the number of qubits where a or b is one.
- **Why $X(a)$ with $a \in \mathcal{C}_2$ fixes $|x\rangle_L$.** $X(a)$ sends $|K^\top x \oplus S^\top y\rangle$ to another term of the same sum, so it only permutes the terms.
- **Why $Z(b)$ with $b \in \mathcal{C}_1^\perp$ fixes $|x\rangle_L$.** $Z(b)|c\rangle = (-1)^{b \cdot c}|c\rangle$, and $b \cdot c = 0$ for every $c \in \mathcal{C}_1$.
- **Why they commute.** $X(a)Z(b) = (-1)^{a \cdot b} Z(b)X(a)$, and $a \cdot b = 0$ because $a \in \mathcal{C}_2 \subseteq \mathcal{C}_1$ and $b \in \mathcal{C}_1^\perp$.
- **Counting.** $k = \dim \mathcal{C}_1 - \dim \mathcal{C}_2$. There are s X-type generators and $n - k - s$ Z-type generators, $n - k$ in all.
- **Notation in the literature.** Nielsen and Chuang, Camps-Moreno et al. and our paper write the CSS code of $\mathcal{C}_2 \subseteq \mathcal{C}_1$ as here. Our paper writes the logical state as $K^\top x + S^\top y$. The slides write the same vector as $K^\top x \oplus S^\top y$, with $\oplus$ for addition mod 2, to keep it apart from the integer sums in signed weights.
- **If asked "is every stabilizer code CSS?"** No. In a CSS code every stabilizer generator is all X or all Z. All codes in this talk are CSS.

Slide 5. The $[\![8, 3, 2]\!]$ code



Clock 4:15–5:45 · **Budget** 90 s · **Clicks** 4

On screen: the matrix $G = [K; S]$ with rows $K_1 = u_1$, $K_2 = u_2$, $K_3 = u_3$ and $S = \mathbf{1}$, columns labelled 000 to 111, $\mathcal{C}_1 = \mathrm{RM}(1, 3)$, and $X(\mathbf{1})$ on the cube. The clicks add the logical basis with its two terms for $x = 100$ and for $x = 111$, then a single Z error, then a weight-two Z logical on an edge with d_X, d_Z and d.

Here is the code we use for the whole first half, on all eight points of the cube. The generator matrix has the logical rows K_1, K_2, K_3, which are the faces u_1, u_2, u_3, and one stabilizer row, all ones. So the only X-type stabilizer generator is $X(\mathbf{1})$, on the left. $\mathcal{C}_1$ is the Reed–Muller code RM(1, 3), the functions of degree at most one, an $[8, 4, 4]$ code, and $k = 3$.

[click] $K^\top x$ is the sum of the logical rows picked by x. Each logical state is that row and its complement, because S is the single all-ones row. For $x = 100$ the pair is $K_1 = 00001111$, in blue, and 11110000, in gold.

[click] For $x = 111$ the pair is $K_1 \oplus K_2 \oplus K_3 = 01101001$ and its complement.

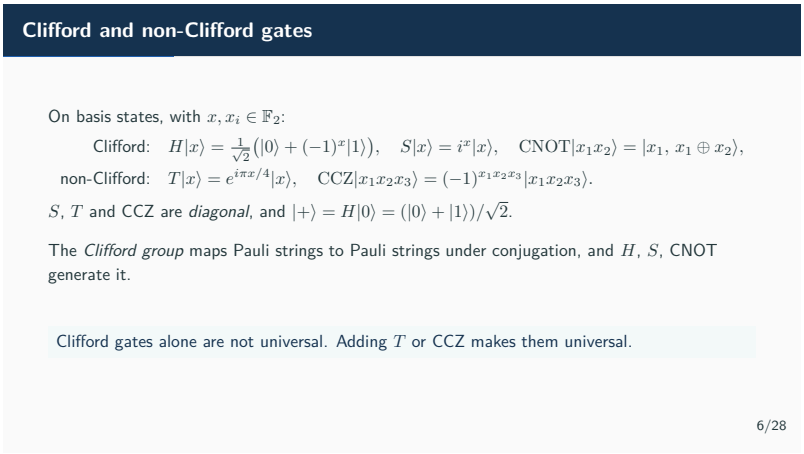
[click] Now a syndrome. Z on the qubit 000 anticommutes with $X(\mathbf{1})$, so it flips its sign: syndrome 1. This error is detected.

[click] Z on the edge from 000 to 100 flips the sign twice: no syndrome. It multiplies $|x\rangle_L$ by $(-1)^{x_1}$, so it is a nontrivial logical operator. Nontrivial X-type and Z-type logical operators have least weights $d_X = 4$, the minimum weight of RM(1, 3), and $d_Z = 2$. So $d = \min(d_X, d_Z) = 2$. *(pause)* We want $d_Z \geq 3$.

Background (not spoken)

- **Why $d_X = 4$.** An X-type logical operator is $X(v)$ with $v \in \mathcal{C}_1 \setminus \mathcal{C}_2$. Every such v is a nonconstant affine function, of weight 4.
- **Why the edge.** Its two columns of S are equal (both 1), so their syndromes cancel. Slide 16 shows that distinct nonzero columns prevent exactly this.
- **Source.** This is the eight-qubit example of Campbell and Howard.

Slide 6. Clifford and non-Clifford gates



Clock 5:45–6:45 · **Budget** 60 s · **Clicks** 2

On screen: H, S, CNOT, T and CCZ, each written by its action on basis states, and the definition of a diagonal gate.

To compute on encoded qubits we also need gates. Each gate is written by what it does to a basis state $|x\rangle$, with x a bit. The first row is H, S and CNOT, the second row T and CCZ. CCZ multiplies $|x_1x_2x_3\rangle$ by $(-1)^{x_1x_2x_3}$, so it puts a minus sign on $|111\rangle$ and nothing else. S, T and CCZ are diagonal, and $|+\rangle = H|0\rangle$.

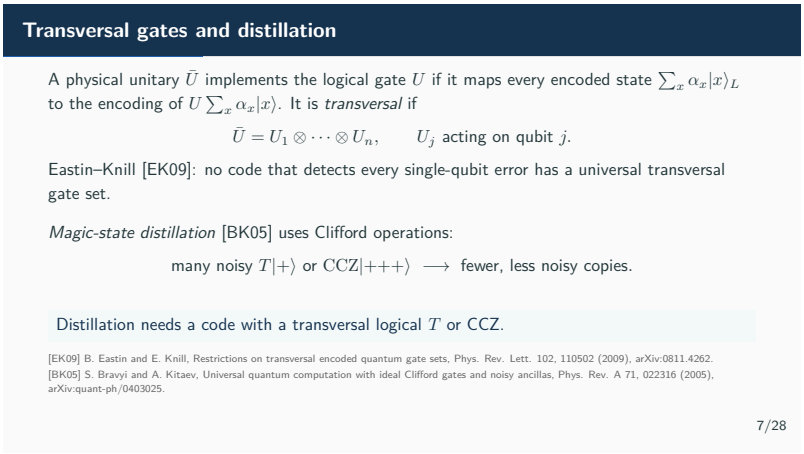
[click] The Clifford group maps Pauli strings to Pauli strings under conjugation. H, S and CNOT generate it. That is the first row. T and CCZ, in the second row, are not Clifford.

[click] Clifford gates alone are not universal. Adding T or CCZ makes them universal.

Background (not spoken)

- **Clifford group.** The unitaries U with $UPU^\dagger$ a Pauli string, up to phase, for every Pauli string P. For example, $HXH = Z$, $SXS^\dagger = Y$ with $Y = iXZ$, and CNOT maps $X \otimes I$ to $X \otimes X$.
- **Why T is not Clifford.** $TXT^\dagger = (X + Y)/\sqrt{2}$, which is not a Pauli string up to phase. For CCZ, conjugating X on the first qubit gives X times a controlled-Z on the other two, which is not a Pauli string.
- **Why Clifford circuits are not universal.** By the Gottesman–Knill theorem, a Clifford circuit on stabilizer states with Pauli measurements can be simulated efficiently on a classical computer.
- **Universality.** Clifford + T generates a dense subgroup of the unitary group up to phase. CCZ with H gives Toffoli, and Clifford + CCZ is also universal.
- **CNOT convention.** $|x_1, x_1 \oplus x_2\rangle$: the first qubit is the control.
- **If asked "why CCZ rather than T?"** The standard Clifford + T circuit for CCZ uses seven T gates. Campbell and Howard distill CCZ states directly, which can lower the cost.

Slide 7. Transversal gates and distillation



Clock 6:45–7:30 · **Budget** 45 s · **Clicks** 2

On screen: the definition of a logical gate and of a transversal gate. The clicks add Eastin–Knill, then magic-state distillation with the takeaway.

The simplest logical gate puts one gate on each qubit. A physical unitary $\bar{U}$ implements the logical gate U if it maps every encoded state $\sum_x \alpha_x |x\rangle_L$ to the encoding of $U \sum_x \alpha_x |x\rangle$. It is transversal if $\bar{U} = U_1 \otimes \cdots \otimes U_n$, with U_j acting on qubit j.

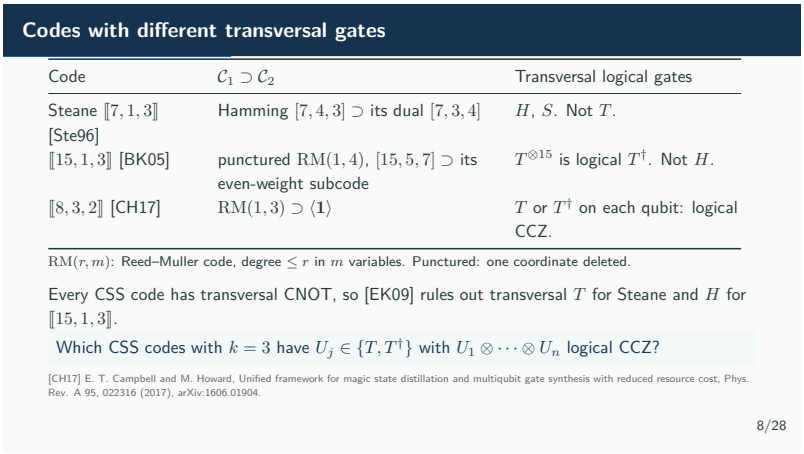
[click] By Eastin and Knill, no code that detects every single-qubit error has a universal transversal gate set. So at least one gate needs another method.

[click] Magic-state distillation, from Bravyi and Kitaev, uses Clifford operations to turn many noisy copies of $T|+\rangle$ or $\mathrm{CCZ}|{+}{+}{+}\rangle$ into fewer, less noisy copies. Distillation needs a code with a transversal logical T or CCZ.

Background (not spoken)

- **The idea behind Eastin–Knill.** The transversal logical gates form a group. If the code detects every single-qubit error, a single-qubit unitary close to the identity acts trivially on the code space. So the group of transversal logical gates is finite, and a finite group cannot be dense in the unitary group.
- **Magic states.** $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, $|T\rangle = T|+\rangle$ and $|\mathrm{CCZ}\rangle = \mathrm{CCZ}|{+}{+}{+}\rangle$. Gate teleportation consumes one such state, together with Clifford gates and measurement, to apply T or CCZ.
- **Why transversal.** A single faulty gate damages only one qubit of the block.

Slide 8. Codes with different transversal gates



Codes with different transversal gates

Code	$\mathcal{C}_1 \supset \mathcal{C}_2$	Transversal logical gates
Steane $[\![7,1,3]\!]$ [Ste96]	Hamming $[7,4,3] \supset$ its dual $[7,3,4]$	H, S. Not T.
$[\![15,1,3]\!]$ [BK05]	punctured $\mathrm{RM}(1,4)$, $[15,5,7] \supset$ its even-weight subcode	$T^{\otimes 15}$ is logical $T^\dagger$. Not H.
$[\![8,3,2]\!]$ [CH17]	$\mathrm{RM}(1,3) \supset \langle \mathbf{1} \rangle$	T or $T^\dagger$ on each qubit: logical CCZ.

$\mathrm{RM}(r,m)$: Reed–Muller code, degree $\leq r$ in m variables. Punctured: one coordinate deleted.

Every CSS code has transversal CNOT, so [EK09] rules out transversal T for Steane and H for $[\![15,1,3]\!]$.

Which CSS codes with $k = 3$ have $U_j \in \{T, T^\dagger\}$ with $U_1 \otimes \cdots \otimes U_n$ logical CCZ?

[CH17] E. T. Campbell and M. Howard, Unified framework for magic state distillation and multiqubit gate synthesis with reduced resource cost, Phys. Rev. A 95, 022316 (2017), arXiv:1606.01904.

8/28

Clock 7:30–8:35 · **Budget** 65 s · **Clicks** 2

On screen: a table of three codes, their pair $\mathcal{C}_1 \supset \mathcal{C}_2$, and their transversal logical gates. The clicks add the Eastin–Knill consequence, then the question of the talk.

Here are three codes. The Steane code comes from the Hamming code and its dual, and H and S are transversal, but T is not. The fifteen-qubit code comes from punctured $\mathrm{RM}(1,4)$ and its even-weight subcode, and T on every qubit is logical $T^\dagger$, but H is not transversal. The $[\![8,3,2]\!]$ code from slide 5 comes from $\mathrm{RM}(1,3)$ and the all-ones row, and T or $T^\dagger$ on each qubit gives logical CCZ.

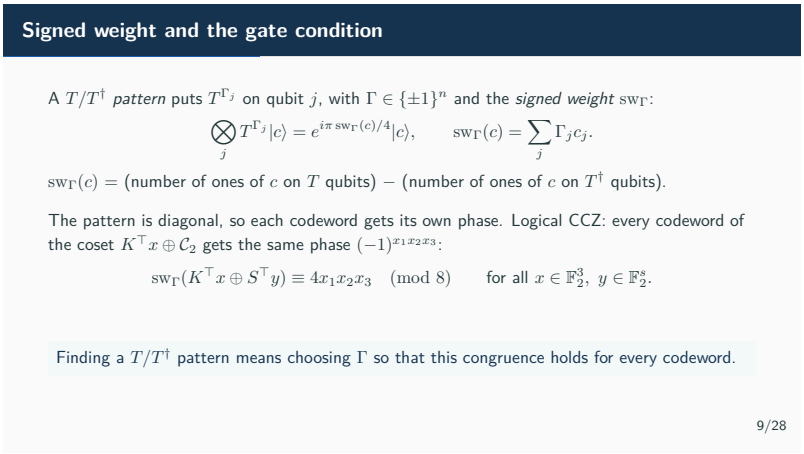
[click] Every CSS code has transversal CNOT, so Eastin–Knill rules out transversal T for Steane and transversal H for the fifteen-qubit code.

[click] The question of this talk. *(pause)* Which CSS codes with $k = 3$ have T or $T^\dagger$ on each qubit, with the product logical CCZ?

Background (not spoken)

- **Steane S.** $S^{\otimes 7}$ multiplies $|c\rangle$ by $i^{|c|}$. Terms of $|0\rangle_L$ have weights $0, 4$ and terms of $|1\rangle_L$ have weights $3, 7$, so $S^{\otimes 7}$ is logical $S^\dagger$, and $(S^\dagger)^{\otimes 7}$ is logical S.
- **Steane H.** $H^{\otimes 7}$ maps $X(a)$ to $Z(a)$. Because $\mathcal{C}_1^\perp = \mathcal{C}_2$, it maps stabilizers to stabilizers and swaps $X(\mathbf{1})$ and $Z(\mathbf{1})$, so it is logical H.
- **Fifteen-qubit T.** The terms of $|0\rangle_L$ have weights $0, 8$ and those of $|1\rangle_L$ have weights $7, 15$, so $T^{\otimes 15}$ gives phase $e^{7i\pi/4}$ on $|1\rangle_L$: logical $T^\dagger$.
- **CNOT.** CNOT between corresponding qubits of two blocks is logical CNOT for every CSS code.
- **If asked about "single-qubit".** The gates U_j may differ from qubit to qubit. The $[\![8,3,2]\!]$ code uses T on some qubits and $T^\dagger$ on others.

Slide 9. Signed weight and the gate condition



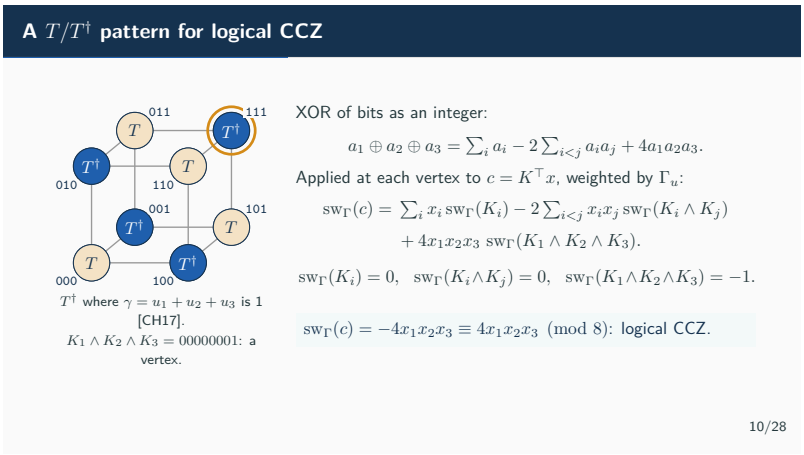
Clock 8:45–9:45 · **Budget** 60 s · **Clicks** 1

On screen: the phase of a $T/T^\dagger$ pattern on a basis state, and the signed weight sw_Γ.

Back to the $[\![8, 3, 2]\!]$ code from slide 5. A $T/T^\dagger$ pattern multiplies each basis state by a phase set by a signed weight. It puts T^{Γ_j} on qubit j, with $\Gamma_j = \pm 1$, so T^{-1} is $T^\dagger$. It multiplies the basis state $|c\rangle$ by $e^{i\pi \mathrm{sw}_\Gamma(c)/4}$. Here $\mathrm{sw}_\Gamma(c) = \sum_j \Gamma_j c_j$ is the signed weight: the number of ones of c on T qubits minus the number of ones of c on $T^\dagger$ qubits.

[click] The pattern is diagonal, so each codeword gets its own phase. For logical CCZ, every codeword of the coset $K^\top x \oplus \mathcal{C}_2$ gets the same phase, $(-1)^{x_1x_2x_3}$. In signed weights, $\mathrm{sw}_\Gamma(K^\top x \oplus S^\top y) \equiv 4x_1x_2x_3 \pmod 8$ for all x and y. So finding a $T/T^\dagger$ pattern means choosing Γ so that this congruence holds for every codeword.

Slide 10. A $T/T^\dagger$ pattern for logical CCZ



Clock 9:45–11:45 · **Budget** 120 s · **Clicks** 4

On screen: the blank cube, the integer expansion of $a_1 \oplus a_2 \oplus a_3$, and $\mathrm{sw}_\Gamma(c)$ in terms of overlaps. The clicks show the pattern with $T^\dagger$ where $\gamma = u_1 + u_2 + u_3$ is one, then a face, an edge and a vertex.

Now we choose signs that meet this condition on the $[\![8, 3, 2]\!]$ code. For three bits, XOR has an integer expansion: $a_1 \oplus a_2 \oplus a_3 = \sum_i a_i - 2\sum_{i<j} a_i a_j + 4a_1a_2a_3$. Apply it at each vertex to the bits of $c = K^\top x$, weight by Γ_u, and sum. That gives $\mathrm{sw}_\Gamma(c)$ in terms of the signed weights of the rows K_i, of their pairwise overlaps, and of the triple overlap. For CCZ, only the last term should survive. *(ask, then wait 7 seconds)* Where would you put the $T^\dagger$ gates? *(Fallback: where $u_1 + u_2 + u_3$ is one.)*

[click] Campbell and Howard put $T^\dagger$ where $\gamma = u_1 + u_2 + u_3$ is one, and T elsewhere.

[click] Row $K_1 = 00001111$ is the face $u_1 = 1$. It has T at 101 and 110, and $T^\dagger$ at 100 and 111. So $\mathrm{sw}_\Gamma(K_1) = 0$, and the same holds for K_2 and K_3.

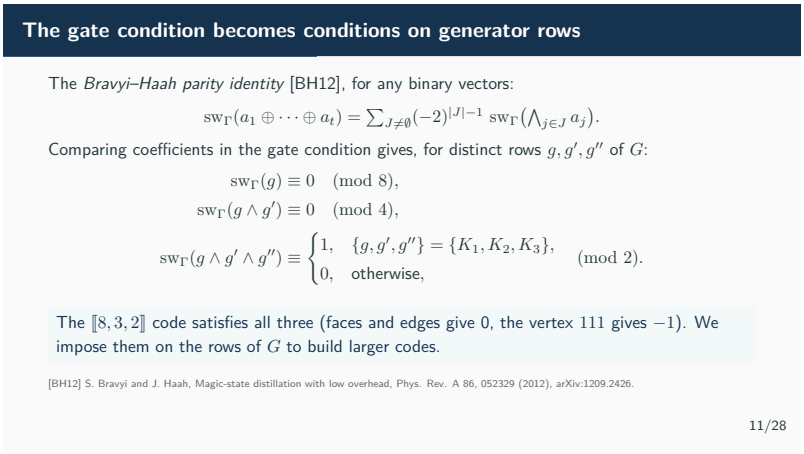
[click] By the dictionary from slide 2, the overlap of rows K_1 and K_2 is the row of u_1u_2, 00000011, the edge $\{110, 111\}$. That is one T and one $T^\dagger$, so its signed weight is 0, and likewise for the other pairs.

[click] The overlap of all three rows is 00000001, the vertex 111, which carries $T^\dagger$. So its signed weight is -1, and $\mathrm{sw}_\Gamma(c) = -4x_1x_2x_3 \equiv 4x_1x_2x_3 \pmod 8$: logical CCZ, read off a face, an edge and a vertex. The stabilizer row **1** adds terms of the same kind, a face and an edge, and they vanish too.

Background (not spoken)

- **A second check.** With $\Gamma_u = (-1)^{\gamma(u)}$, $\mathrm{sw}_\Gamma(c) = |c| - 2|\gamma \wedge c| = |\gamma \oplus c| - 4$. Every codeword other than γ and its complement is at distance 4 from γ, so it gets phase 1. The two codewords of the label 111 are at distance 0 and 8, so both get phase -1.
- **Source.** This is the eight-qubit example of Campbell and Howard.

Slide 11. The gate condition becomes conditions on generator rows



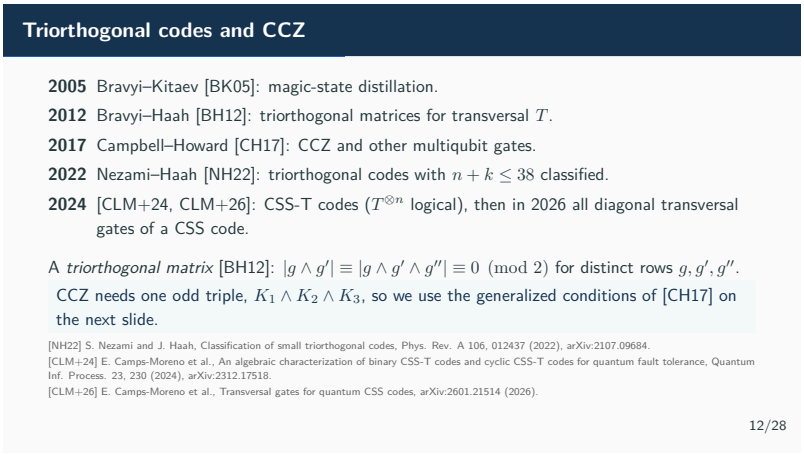
Clock 11:45–12:30 · **Budget** 45 s · **Clicks** 1

On screen: the Bravyi–Haah parity identity, and the three congruences for single rows, pairs and triples.

The expansion we just used holds for any number of vectors. This is the Bravyi–Haah parity identity: an overlap of t vectors has coefficient $(-2)^{t-1}$. Comparing coefficients in the gate condition gives three congruences, for distinct rows g, g', g'' of G. Single rows need $\mathrm{sw}_\Gamma(g) \equiv 0 \pmod 8$. Pair overlaps carry -2, so they need $\mathrm{sw}_\Gamma(g \wedge g') \equiv 0 \pmod 4$. Triple overlaps carry 4, so only their parity matters: odd exactly for the triple K_1, K_2, K_3, and even otherwise. Overlaps of four or more rows carry a multiple of eight and drop out.

[click] The $[\![8, 3, 2]\!]$ code satisfies all three: faces and edges give 0, and the vertex 111 gives -1. We impose these three congruences on the rows of G to build larger codes.

Slide 12. Triorthogonal codes and CCZ



Clock 12:40–13:40 · **Budget** 60 s · **Clicks** 1

On screen: a timeline from 2005 to 2026. The click adds the triorthogonal matrix condition and the need for one odd triple.

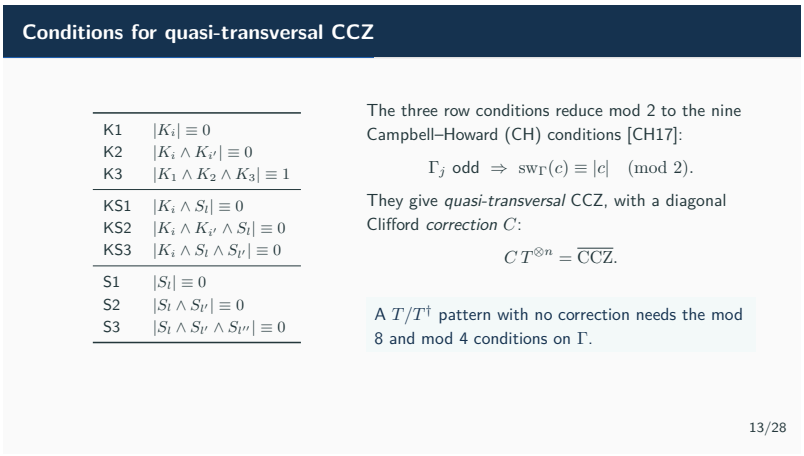
Briefly, the history. Bravyi and Kitaev introduced magic-state distillation. Bravyi and Haah wrote transversal T as parity conditions on binary rows, triorthogonal matrices. Campbell and Howard gave the CCZ version, which we use. Nezami and Haah classified small triorthogonal codes. Camps-Moreno and coauthors study CSS-T codes, on which T on every qubit is a logical gate, and in 2026 all diagonal transversal gates of a CSS code.

[click] A triorthogonal matrix has even pair and triple overlaps for distinct rows. CCZ needs one odd triple, $K_1 \wedge K_2 \wedge K_3$, so it needs the generalized conditions of Campbell and Howard on the next slide.

Background (not spoken)

- **Odd and even rows.** In a triorthogonal matrix, odd-weight rows give logical X operators and even-weight rows give X-type stabilizers.
- **Triorthogonal space.** Nezami and Haah call a subspace $\mathcal{H}$ triorthogonal if $\sum_j h_j h'_j h''_j \equiv 0 \pmod 2$ for all $h, h', h'' \in \mathcal{H}$, repeats allowed. So every vector has even weight, and the row span of a matrix with odd rows is not such a space. Unital means $\mathbf{1} \in \mathcal{H}$. They classify these spaces and obtain codes by deleting (puncturing) coordinates.
- **CSS-T.** A CSS code on which T on every qubit acts as a logical gate.
- **If asked about $n + k \leq 38$.** Nezami and Haah classify triorthogonal codes with $n + k \leq 38$, a range for T codes. It is a different statement from our bound $n \leq 38$ for CCZ.

Slide 13. Conditions for quasi-transversal CCZ



Clock 13:40–15:10 · **Budget** 90 s · **Clicks** 2

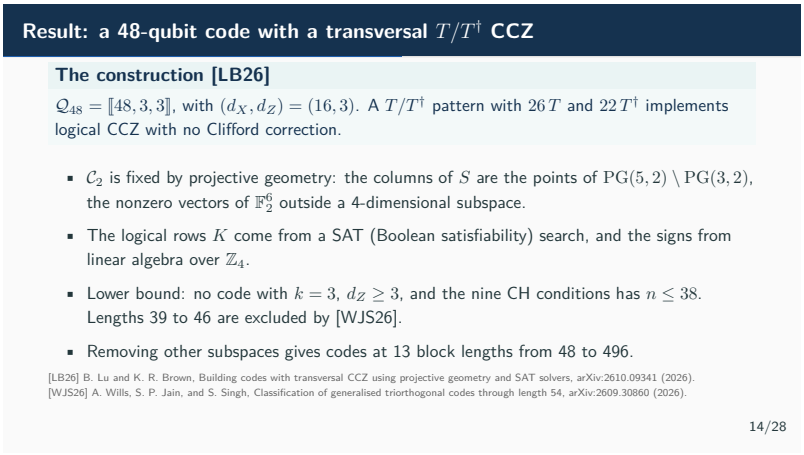
On screen: the nine parity conditions K1–K3, KS1–KS3 and S1–S3.

Reduce the three signed conditions mod 2. Each Γ_j is odd, so a signed weight has the same parity as the ordinary weight. What remains are these nine parity conditions of Campbell and Howard, the CH conditions. K1 to K3 involve only logical rows, KS1 to KS3 mix logical and stabilizer rows, and S1 to S3 involve only stabilizer rows. K3 is the only odd condition, and it selects CCZ.

[click] These nine conditions make T on every qubit, followed by some diagonal Clifford gate C, act as logical CCZ. C is called the correction, and this is quasi-transversal CCZ.

[click] Here is a distinction I want to keep clear. *(pause)* A $T/T^\dagger$ pattern with no correction needs more, namely the signed congruences mod 8 and mod 4. Those are conditions on the signs Γ as well as on the matrix. A code can satisfy all nine parities and still have no valid choice of signs. We will see an example at length 47.

Slide 14. Result: a 48-qubit code with a transversal $T/T^\dagger$ CCZ

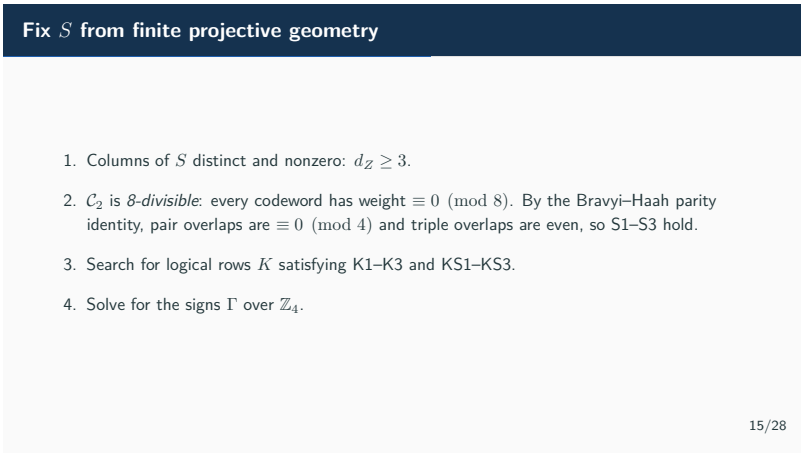


Clock 15:20–16:40 · **Budget** 80 s · **Clicks** 0

On screen: $\mathcal{Q}_{48} = [\![48, 3, 3]\!]$ with $(d_X, d_Z) = (16, 3)$, and four bullets on the construction and the bounds.

Here is the result. $\mathcal{Q}_{48}$ is a $[\![48, 3, 3]\!]$ code, with $d_X = 16$ and $d_Z = 3$. A pattern of 26 T gates and 22 $T^\dagger$ gates implements logical CCZ, with no Clifford correction. It is built in three steps. Geometry fixes the stabilizers: the columns of S are the points of $\mathrm{PG}(5,2) \setminus \mathrm{PG}(3,2)$, the nonzero vectors of $\mathbb{F}_2^6$ outside a 4-dimensional subspace. A SAT search, a Boolean satisfiability solver, finds the logical rows K. And linear algebra over $\mathbb{Z}_4$ finds the signs. *(pause)* For the lower bound, our theorem says that no code with three logical qubits, $d_Z \geq 3$, and the nine CH conditions has $n \leq 38$. Lengths 39 to 46 are excluded by the classification of Wills, Jain and Singh. Removing other subspaces gives codes at 13 block lengths from 48 to 496.

Slide 15. Fix S from finite projective geometry



Clock 16:40–17:40 · **Budget** 60 s · **Clicks** 3

On screen: step 1, distinct nonzero columns of S give $d_Z \geq 3$.

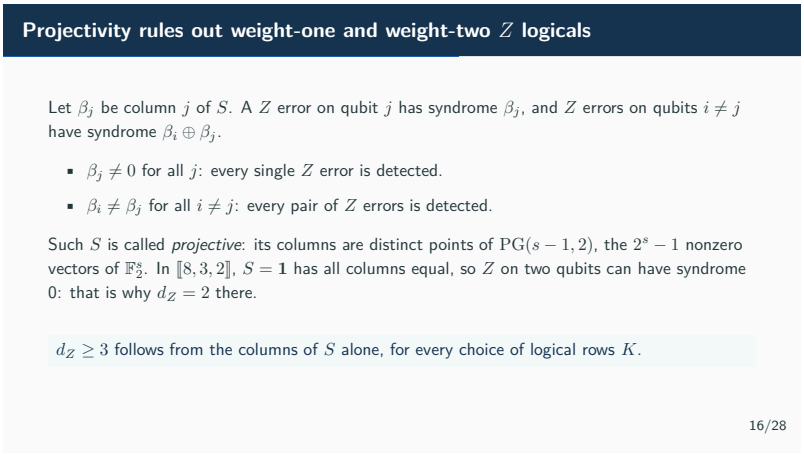
The construction has four steps. First, choose S with distinct nonzero columns. That alone gives $d_Z \geq 3$, as the next slide shows.

[click] Second, require $\mathcal{C}_2$ to be 8-divisible: every codeword has weight divisible by 8. By the Bravyi–Haah parity identity, pair overlaps are then zero mod 4 and triple overlaps are even, so S1 to S3 hold. This divisibility is our design choice. The CH conditions do not require it.

[click] Third, search for logical rows K that satisfy K1 to K3 and KS1 to KS3. This is the only search.

[click] Fourth, solve for the signs Γ over $\mathbb{Z}_4$. That is a linear-algebra test.

Slide 16. Projectivity rules out weight-one and weight-two Z logicals

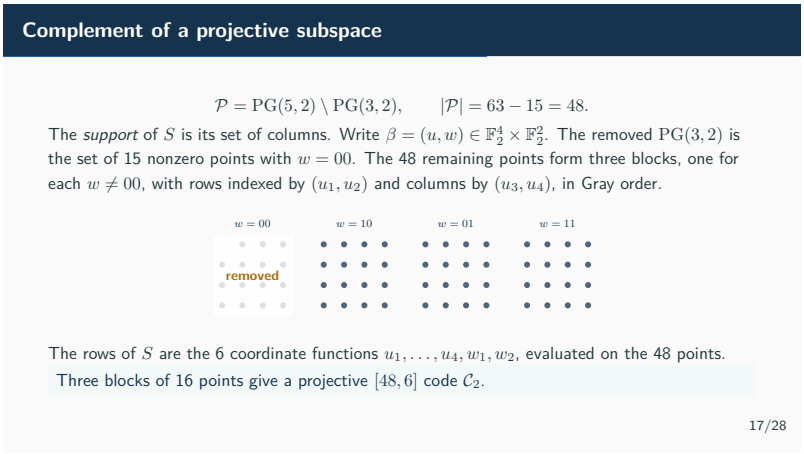


Clock 17:40–18:40 · **Budget** 60 s · **Clicks** 0

On screen: the syndrome of a Z error is a column β_j of S, and distinct nonzero columns detect one or two Z errors.

Step 1 asked for distinct nonzero columns. Here is why that gives $d_Z \geq 3$. Let β_j be column j of S. A Z error on qubit j has syndrome β_j, and Z errors on qubits $i \neq j$ have syndrome $\beta_i \oplus \beta_j$. If no column is zero, every single Z error is detected. If the columns are distinct, $\beta_i \oplus \beta_j \neq 0$, so every pair of Z errors is detected too. So $d_Z \geq 3$. *(pause)* Such an S is called projective: its columns are distinct points of $\mathrm{PG}(s-1,2)$, the 2^s-1 nonzero vectors of $\mathbb{F}_2^s$. In the $[\![8,3,2]\!]$ code, $S = \mathbf{1}$ has all columns equal, so Z on two qubits can have syndrome zero: that is why $d_Z = 2$ there. For a projective S, $d_Z \geq 3$ follows from the columns of S alone, for every choice of logical rows K. It covers d_Z only, and we compute d_X for the final code.

Slide 17. Complement of a projective subspace



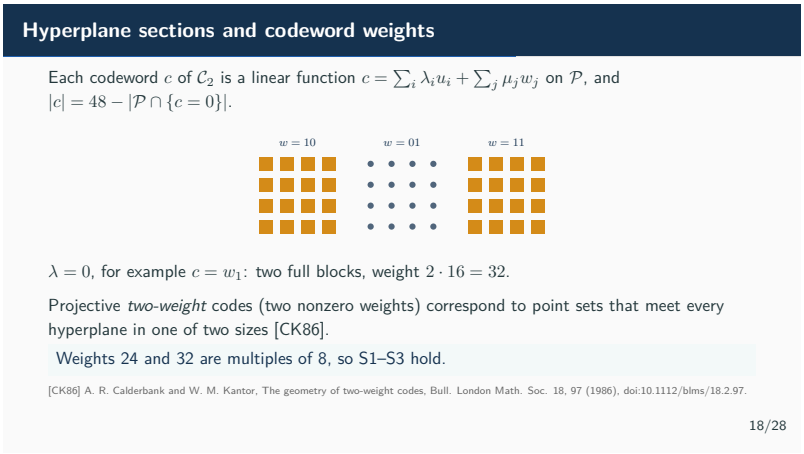
Clock 18:40–19:40 · **Budget** 60 s · **Clicks** 1

On screen: $\mathcal{P} = \mathrm{PG}(5,2) \setminus \mathrm{PG}(3,2)$ with $63 - 15 = 48$ points, drawn as four blocks indexed by w.

Here is the projective S for $\mathcal{Q}_{48}$. Take the 63 points of PG(5, 2) and remove a PG(3, 2), which has 15 points. That leaves 48. The support of S is its set of columns. Write a point as $\beta = (u, w)$, with $u \in \mathbb{F}_2^4$ and $w \in \mathbb{F}_2^2$. The removed subspace is the 15 nonzero points with $w = 00$. In the picture, the four blocks are the values of w. Inside each block the rows are indexed by the pair (u_1, u_2) and the columns by (u_3, u_4).

[click] Remove the block $w = 00$. The 48 remaining points form three blocks, one for each $w \neq 00$. The rows of S are the six coordinate functions $u_1, \ldots, u_4, w_1, w_2$, evaluated on the 48 points. Three blocks of 16 points give a projective [48, 6] code, $\mathcal{C}_2$. We keep this picture for the weights, the logical rows and the signs.

Slide 18. Hyperplane sections and codeword weights



Clock 19:40–21:10 · **Budget** 90 s · **Clicks** 1

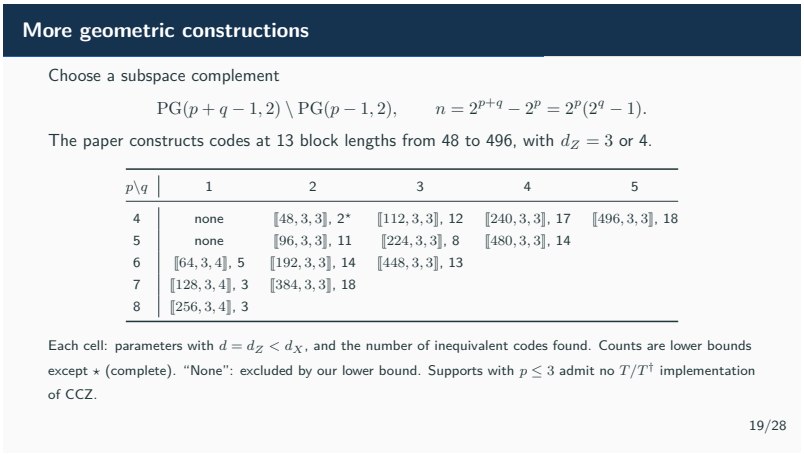
On screen: a codeword of $\mathcal{C}_2$ as a linear function on the 48 points, with u_1 marked in gold.

Step 2 needs the weights of $\mathcal{C}_2$. Each codeword of $\mathcal{C}_2$ is a linear function $c = \sum_i \lambda_i u_i + \sum_j \mu_j w_j$ on the 48 points. Its weight is 48 minus the number of points on the hyperplane where it vanishes. If $\lambda \neq 0$, for example $c = u_1$, the gold squares show that it is one on eight points of each block. So the weight is $3 \cdot 8 = 24$.

(ask, then wait 7 seconds) What is the weight of w_1? *(Fallback: click and read it off.)*

[click] w_1 is one on two full blocks, so its weight is $2 \cdot 16 = 32$. These are the only two cases. So the nonzero weights are 24 and 32, both multiples of 8, and S1 to S3 hold. *(pause)* A projective code with two nonzero weights is a two-weight code. Calderbank and Kantor show these codes correspond to point sets that meet every hyperplane in one of two sizes.

Slide 19. More geometric constructions



More geometric constructions

Choose a subspace complement

$$\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2), \qquad n = 2^{p+q} - 2^p = 2^p(2^q-1).$$

The paper constructs codes at 13 block lengths from 48 to 496, with $d_Z = 3$ or 4.

$p\backslash q$	1	2	3	4	5
4	none	$[\![48,3,3]\!]$, 2*	$[\![112,3,3]\!]$, 12	$[\![240,3,3]\!]$, 17	$[\![496,3,3]\!]$, 18
5	none	$[\![96,3,3]\!]$, 11	$[\![224,3,3]\!]$, 8	$[\![480,3,3]\!]$, 14	
6	$[\![64,3,4]\!]$, 5	$[\![192,3,3]\!]$, 14	$[\![448,3,3]\!]$, 13		
7	$[\![128,3,4]\!]$, 3	$[\![384,3,3]\!]$, 18			
8	$[\![256,3,4]\!]$, 3				

Each cell: parameters with $d = d_Z < d_X$, and the number of inequivalent codes found. Counts are lower bounds except $\star$ (complete). "None": excluded by our lower bound. Supports with $p \leq 3$ admit no $T/T^\dagger$ implementation of CCZ.

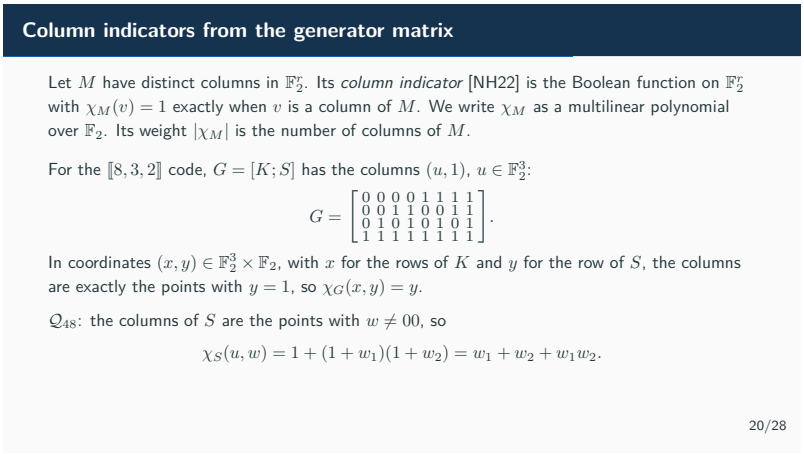
19/28

Clock 21:10–22:10 · **Budget** 60 s · **Clicks** 0

On screen: $n = 2^p(2^q-1)$ for $\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2)$, and a table of codes found for $p = 4$ to 8 and $q = 1$ to 5.

The same idea works for any subspace complement, $\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2)$. The length is $n = 2^p(2^q-1)$. $\mathcal{Q}_{48}$ is the case $p = 4$, $q = 2$. The paper constructs codes at 13 block lengths from 48 to 496, with $d_Z = 3$ or 4. Each cell gives the parameters and the number of inequivalent codes we found. For example, $p = 6$ and $q = 1$ gives 64 qubits with $d_Z = 4$. *(pause)* Two qualifications. Only the starred entry at 48 is a complete census for that support. The other counts are lower bounds. The entries marked "none" are excluded by our lower bound, and supports with $p \leq 3$ admit no $T/T^\dagger$ implementation of CCZ. This is a family of finite constructions, not an asymptotic statement.

Slide 20. Column indicators from the generator matrix



Clock 22:20–23:40 · **Budget** 80 s · **Clicks** 2

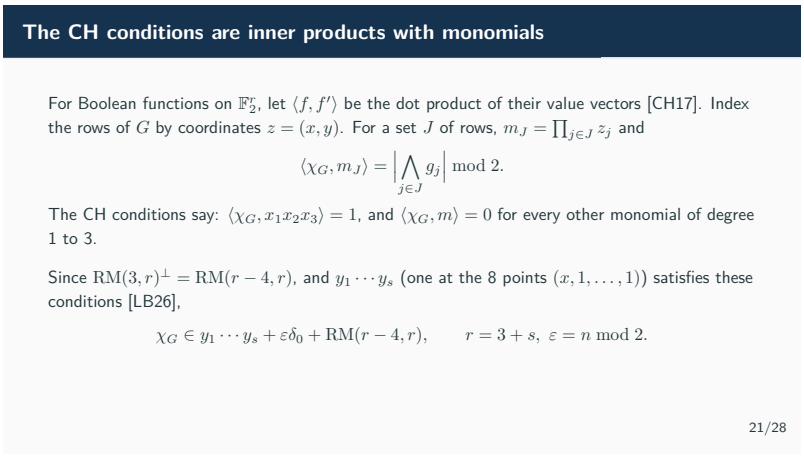
On screen: the definition of the column indicator χ_M.

So far we read rows. To find the logical rows of a larger code we switch to columns. Let M have distinct columns in $\mathbb{F}_2^r$. Its column indicator, as in Nezami and Haah, is the Boolean function χ_M on $\mathbb{F}_2^r$ with $\chi_M(v) = 1$ exactly when v is a column of M. We write it as a multilinear polynomial over $\mathbb{F}_2$. Its weight is the number of columns of M.

[click] For the $[\![8, 3, 2]\!]$ code, $G = [K; S]$ has the columns $(u, 1)$ for every $u \in \mathbb{F}_2^3$. In coordinates (x, y), with x for the rows of K and y for the row of S, every x occurs with $y = 1$, and no column has $y = 0$. So $\chi_G(x, y) = y$.

[click] For $\mathcal{Q}_{48}$, the columns of S are the points with $w \neq 00$. So $\chi_S(u, w) = 1 + (1 + w_1)(1 + w_2) = w_1 + w_2 + w_1 w_2$. It has degree two and does not depend on u.

Slide 21. The CH conditions are inner products with monomials



Clock 23:40–25:10 · **Budget** 90 s · **Clicks** 2

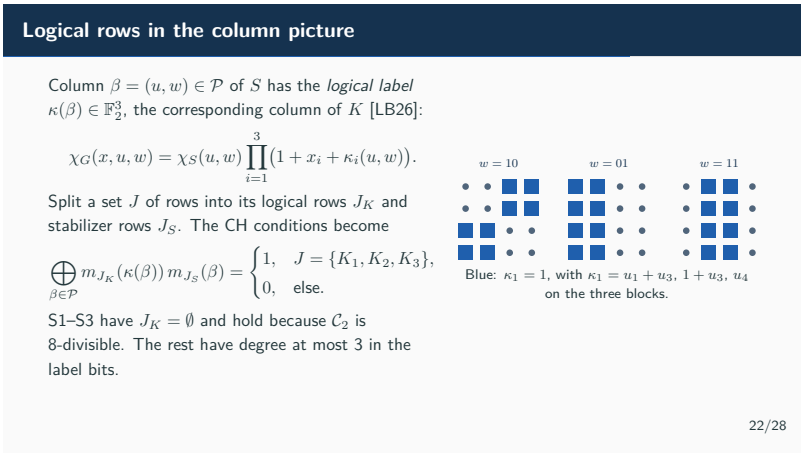
On screen: the inner product $\langle f, f' \rangle$ and the identity $\langle \chi_G, m_J \rangle = |\bigwedge_{j \in J} g_j| \bmod 2$.

Columns help because overlaps of rows become inner products with monomials. For Boolean functions on $\mathbb{F}_2^r$, $\langle f, f' \rangle$ is the dot product of their value vectors, mod 2. Index the rows of G by coordinates $z = (x, y)$. For a set J of rows, let $m_J = \prod_{j \in J} z_j$ be the monomial. By the dictionary from slide 2, the overlap of rows is a product of coordinates. So $\langle \chi_G, m_J \rangle$ is the weight of the overlap of those rows, mod 2. Summing over the columns is summing over the support of χ_G.

[click] So the CH conditions say this: $\langle \chi_G, x_1 x_2 x_3 \rangle = 1$, and $\langle \chi_G, m \rangle = 0$ for every other monomial of degree one to three.

[click] The dual of $\mathrm{RM}(3, r)$ is $\mathrm{RM}(r-4, r)$. The product $y_1 \cdots y_s$ satisfies these conditions. It is one at the eight points $(x, 1, \ldots, 1)$, and among these monomials only $x_1 x_2 x_3$ has odd inner product with it. So every valid χ_G lies in $y_1 \cdots y_s + \varepsilon \delta_0 + \mathrm{RM}(r-4, r)$, where $r = 3 + s$. Here δ_0 is the weight-one vector at the origin, and $\varepsilon = n \bmod 2$. That coset statement is a lemma in our paper.

Slide 22. Logical rows in the column picture



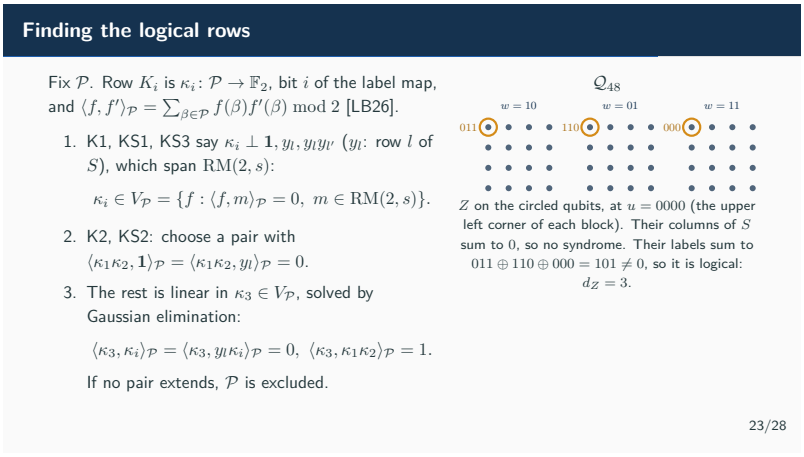
Clock 25:10–26:40 · **Budget** 90 s · **Clicks** 1

On screen: the logical label $\kappa(\beta)$, the factorization of χ_G, and the CH conditions as sums over the 48 points.

For $\mathcal{Q}_{48}$, S is fixed, so the unknowns are the logical rows. In the column picture, each column β of S has a logical label $\kappa(\beta) \in \mathbb{F}_2^3$, the corresponding column of K. Then $\chi_G(x, u, w) = \chi_S(u, w) \prod_{i=1}^{3}(1+x_i+\kappa_i(u, w))$, which forces x to equal κ at each point. Split a set J of rows into its logical rows J_K and stabilizer rows J_S. The CH conditions become a sum over the 48 points, which must equal one exactly for the logical triple. S1 to S3 have no logical rows, and they hold because $\mathcal{C}_2$ is 8-divisible. The rest are equations of degree at most three in the label bits, three per point.

[click] Here is the first logical row of $\mathcal{Q}_{48}$ on the support. κ_1 is $u_1 + u_3$ on the first block, $1 + u_3$ on the second, and u_4 on the third. It has eight ones in each block.

Slide 23. Finding the logical rows



Clock 26:40–28:40 · **Budget** 120 s · **Clicks** 3

On screen: step 1 of Algorithm 1, $\kappa_i \in V_{\mathcal{P}}$, and on the right the support of $\mathcal{Q}_{48}$ with $\dim V_{\mathcal{P}} = 27$. The clicks show κ_1 and κ_2, then κ_3, then the weight-3 Z logical with the labels of its three qubits.

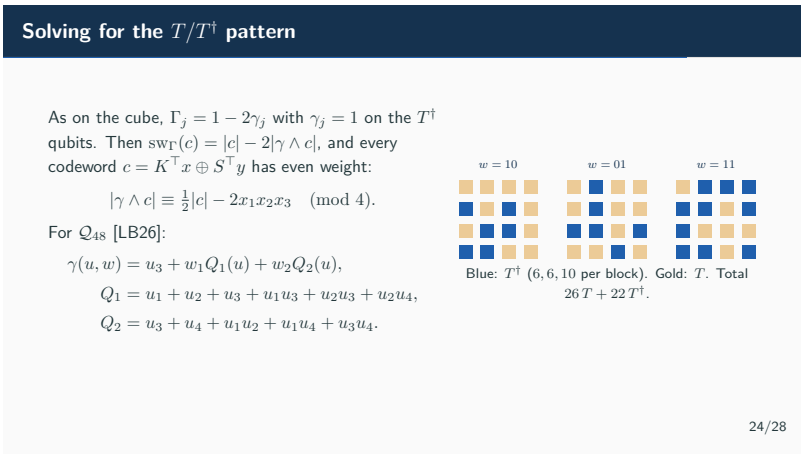
This is Algorithm 1 of the paper. Fix the support $\mathcal{P}$. Row K_i, read as a function on the points of $\mathcal{P}$, is κ_i, the i-th bit of the label map. So $\kappa_1(\beta) = 1$ means K_1 has a one at qubit β. Step one: K1, KS1 and KS3 for a single row say that κ_i is orthogonal to $\mathbf{1}$, to each y_l, and to each product $y_l y_{l'}$. Those functions span $\mathrm{RM}(2, s)$. So κ_i lies in the linear subspace $V_{\mathcal{P}}$. *(pause)* On the right is the support of $\mathcal{Q}_{48}$, where $V_{\mathcal{P}}$ has dimension 27. Its logical rows, found by our SAT search, pass each step.

[click] Step two: choose a pair $\kappa_1, \kappa_2 \in V_{\mathcal{P}}$ whose product is orthogonal to $\mathbf{1}$ and to every y_l. Those are the conditions K2 and KS2. A square is a qubit where K_1 or K_2 has a one. Blue means only K_1, gold means only K_2, and red means both. The red squares are the overlap $K_1 \wedge K_2$. There are 12, an even number, as K2 requires.

[click] Step three: for a fixed pair, the conditions on κ_3 are linear. Gaussian elimination over $\mathbb{F}_2$ finds κ_3 or proves that none exists. The squares now show where K_3 has a one, and the red ones are where K_1 and K_2 also do, the overlap $K_1 \wedge K_2 \wedge K_3$. There are 7, an odd number, as K3 requires.

[click] If no pair extends, the support is excluded. *(pause)* Now the distance. Take the three circled qubits at $u = 0000$, the upper left corner of each block. Their columns of S sum to zero, so Z on them has no syndrome. Their labels, the columns of K printed beside the circles, sum to 101, not zero. So this Z flips the sign of $x_1 \oplus x_3$. It is a logical operator of weight three, and with projectivity $d_Z = 3$.

Slide 24. Solving for the $T/T^\dagger$ pattern



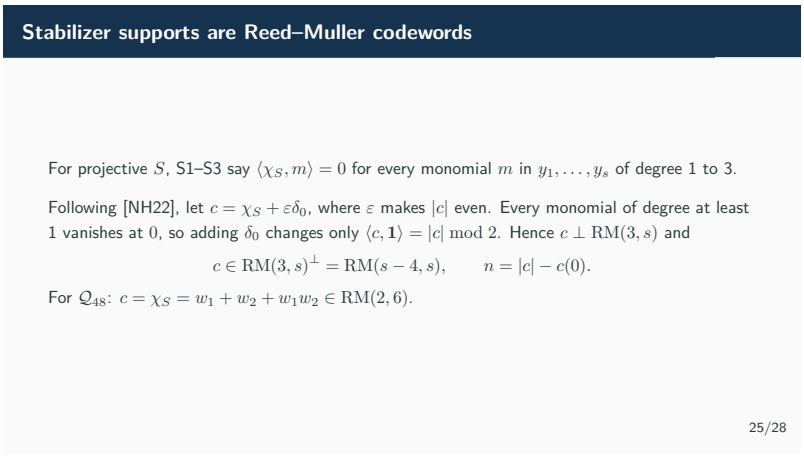
Clock 28:40–30:10 · **Budget** 90 s · **Clicks** 1

On screen: $\Gamma_j = 1 - 2\gamma_j$ and the congruence $|\gamma \wedge c| \equiv \frac{1}{2}|c| - 2x_1x_2x_3 \pmod 4$.

The last step is the signs. As on the cube, write $\Gamma_j = 1 - 2\gamma_j$, with γ_j a bit that is one where we put $T^\dagger$. Then $\mathrm{sw}_\Gamma(c) = |c| - 2|\gamma \wedge c|$, an ordinary integer count. Every codeword has even weight, so we can divide the gate condition by two. The weight of the overlap must satisfy $|\gamma \wedge c| \equiv \frac{1}{2}|c| - 2x_1x_2x_3 \pmod 4$. That is linear in γ over $\mathbb{Z}_4$, with the extra requirement that γ is binary. Elimination over $\mathbb{Z}_4$ decides whether a solution exists.

[click] For $\mathcal{Q}_{48}$, here is the solution from the paper. $\gamma(u,w) = u_3 + w_1Q_1(u) + w_2Q_2(u)$, with two quadratic functions Q_1 and Q_2. In the picture, blue is $T^\dagger$, with 6, 6 and 10 in the three blocks, 22 in all. Gold is T, 26 in all.

Slide 25. Stabilizer supports are Reed–Muller codewords

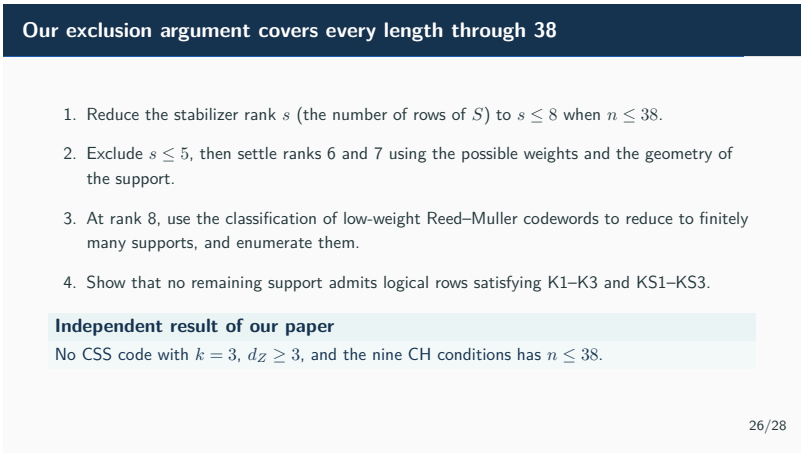


Clock 30:20–32:10 · **Budget** 110 s · **Clicks** 0

On screen: S1–S3 as $\langle \chi_S, m \rangle = 0$ for monomials of degree 1 to 3, and $c = \chi_S + \varepsilon\delta_0 \in \mathrm{RM}(s-4, s)$.

The same column picture explains why short codes fail. Take a projective S. In the column picture, S1 to S3 say that χ_S has inner product zero with every monomial of degree one to three. Following Nezami and Haah, let $c = \chi_S + \varepsilon\delta_0$, with ε chosen to make the weight of c even. Every monomial of positive degree vanishes at the origin. So adding δ_0 changes only the inner product with $\mathbf{1}$, which is the parity of the weight. *(pause)* Hence $c \perp \mathrm{RM}(3, s)$, and $c \in \mathrm{RM}(s-4, s)$. The block length is $n = |c| - c(0)$. For $\mathcal{Q}_{48}$, $c = w_1 + w_2 + w_1 w_2$, a codeword of $\mathrm{RM}(2, 6)$. So a short code needs a low-weight codeword of a Reed–Muller code. Low-weight codewords of Reed–Muller codes are classified, and that is what makes the search below length 39 finite.

Slide 26. Our exclusion argument covers every length through 38

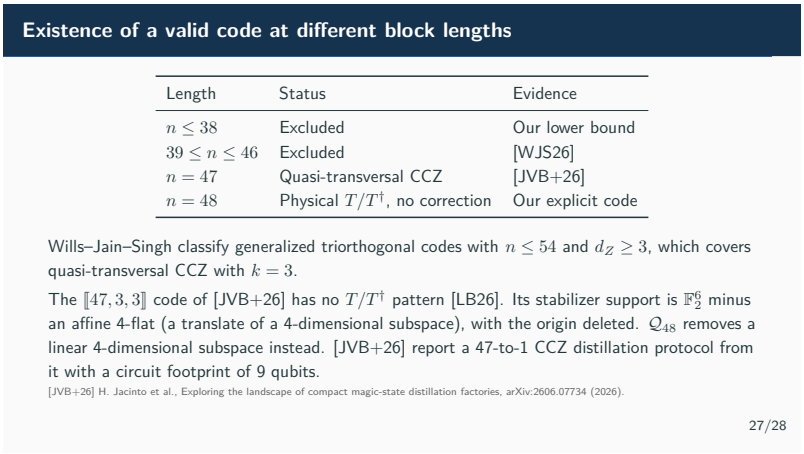


Clock 32:10–33:40 · **Budget** 90 s · **Clicks** 0

On screen: four proof stages, and the theorem: no CSS code with $k = 3$, $d_Z \geq 3$ and the nine CH conditions has $n \leq 38$.

The proof that nothing works through length 38 has four stages. First, reduce the stabilizer rank s, the number of rows of S, to $s \leq 8$ when $n \leq 38$. Second, exclude $s \leq 5$, then settle ranks 6 and 7 using the weight distribution and the geometry of the support. Third, at rank 8, use the classification of low-weight Reed–Muller codewords to reduce to finitely many supports, and enumerate them. Fourth, show that no remaining support admits logical rows satisfying K1 to K3 and KS1 to KS3. *(pause)* The result is our theorem. No CSS code with $k = 3$, $d_Z \geq 3$ and the nine CH conditions has $n \leq 38$. Two remarks. The rank-8 coverage and the logical-row exclusions are complete, not sampled. And because the theorem assumes only the nine parity conditions, it rules out quasi-transversal codes as well as $T/T^\dagger$ patterns.

Slide 27. Existence of a valid code at different block lengths



Clock 33:40–35:40 · **Budget** 120 s · **Clicks** 0

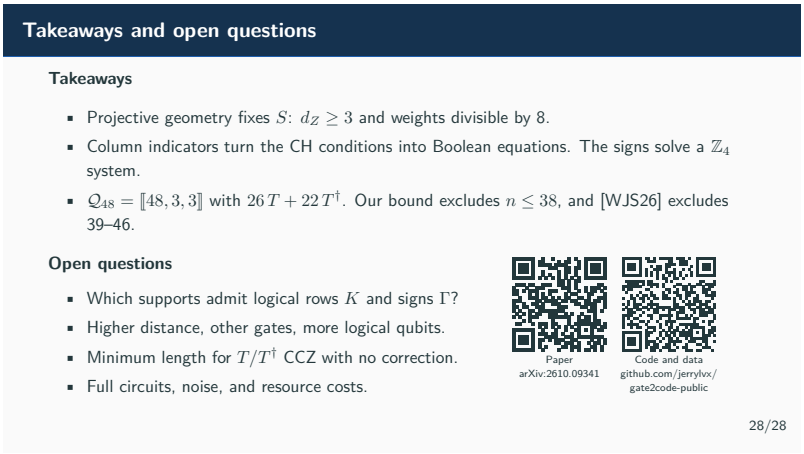
On screen: a table of lengths, status and evidence, and three sentences on the $[\![47, 3, 3]\!]$ code.

Here is the situation by length. Up to 38 is excluded by our bound. 39 to 46 is excluded by Wills, Jain and Singh, who classify generalized triorthogonal codes up to length 54 with $d_Z \geq 3$. That classification covers quasi-transversal CCZ with three logical qubits. At 47 there is a quasi-transversal CCZ code, the $[\![47, 3, 3]\!]$ code of Jacinto and coauthors. At 48 we have $\mathcal{Q}_{48}$, with a $T/T^\dagger$ pattern and no correction. *(pause)* Now the distinction between 47 and 48. The 47-qubit code satisfies the nine parities, but it has no $T/T^\dagger$ pattern. We show this in our paper. So at 47 a Clifford correction is needed. That statement is about this specific code. It does not show that 48 is the minimum length for a $T/T^\dagger$ pattern, and it does not show that the 47-qubit code is unique. It is also not $\mathcal{Q}_{48}$ with one point removed. Its stabilizer support is $\mathbb{F}_2^6$ minus an affine 4-flat, a translate of a 4-dimensional subspace, with the origin deleted. $\mathcal{Q}_{48}$ removes a linear 4-dimensional subspace. Finally, Jacinto and coauthors report a 47-to-1 CCZ distillation protocol from their code with circuit footprint of 9 qubits. That number is theirs, and it is about the circuit, not the code length.

Background (not spoken)

- **If asked why the 47-qubit code is not $\mathcal{Q}_{48}$ punctured.** The stabilizer weights differ: 16, 24 and 32 for the 47-qubit code, and 23, 24, 31 and 32 for any puncture of $\mathcal{Q}_{48}$.

Slide 28. Takeaways and open questions

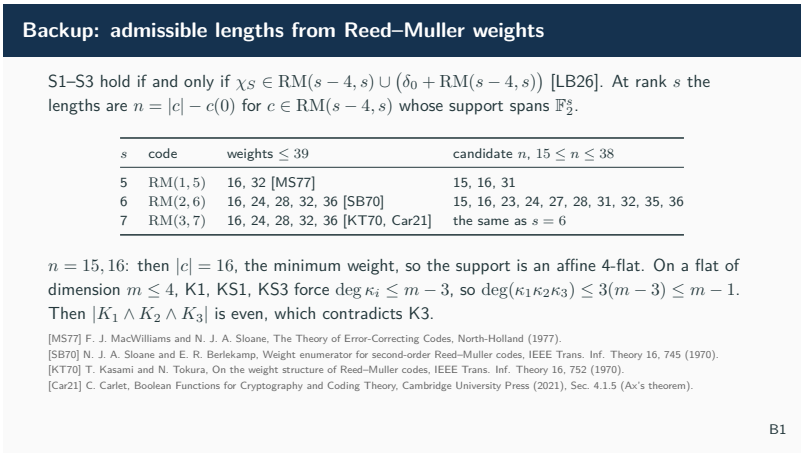


Clock 35:40–37:10 · **Budget** 90 s · **Clicks** 0

On screen: three takeaways, four open questions, and QR codes for the paper and the code.

Here is what we contributed. *(pause)* Projective geometry fixes S, giving $d_Z \geq 3$ and weights divisible by eight. Column indicators turn the CH conditions into Boolean equations for the logical rows, and the signs solve a linear system over $\mathbb{Z}_4$. Together these give $\mathcal{Q}_{48}$, a $[\![48, 3, 3]\!]$ code in which 26 T and 22 $T^\dagger$ gates act as logical CCZ. Our bound excludes lengths up to 38, and Wills, Jain and Singh exclude 39 to 46. In one sentence: geometry fixes the stabilizers, a search finds the logical rows, and linear algebra over $\mathbb{Z}_4$ finds the signs. *(pause)* Some open questions. Which supports admit logical rows and signs? Can we reach higher distance, other gates, or more logical qubits? What is the minimum length for a CCZ pattern of T and $T^\dagger$ with no correction? And what do full circuits cost once noise is included? If you work on divisible codes, or on completing a code from a fixed support, I would like to hear your ideas. The QR codes link to the paper and the code. *(salute, then stop)* It has been a pleasure to present this work to algebraists and coding theorists here at Virginia Tech. I look forward to your questions.

Backup B1. Backup: admissible lengths from Reed–Muller weights



Backup: admissible lengths from Reed–Muller weights

S1–S3 hold if and only if $\chi_S \in \mathrm{RM}(s-4,s) \cup \big(\delta_0 + \mathrm{RM}(s-4,s)\big)$ [LB26]. At rank s the lengths are $n = |c| - c(0)$ for $c \in \mathrm{RM}(s-4,s)$ whose support spans $\mathbb{F}_2^s$.

s	code	weights ≤ 39	candidate n, $15 \leq n \leq 38$
5	$\mathrm{RM}(1,5)$	16, 32 [MS77]	15, 16, 31
6	$\mathrm{RM}(2,6)$	16, 24, 28, 32, 36 [SB70]	15, 16, 23, 24, 27, 28, 31, 32, 35, 36
7	$\mathrm{RM}(3,7)$	16, 24, 28, 32, 36 [KT70, Car21]	the same as $s = 6$

$n = 15, 16$: then $|c| = 16$, the minimum weight, so the support is an affine 4-flat. On a flat of dimension $m \leq 4$, K1, KS1, KS3 force $\deg \kappa_i \leq m-3$, so $\deg(\kappa_1\kappa_2\kappa_3) \leq 3(m-3) \leq m-1$. Then $|K_1 \wedge K_2 \wedge K_3|$ is even, which contradicts K3.

[MS77] F. J. MacWilliams and N. J. A. Sloane, The Theory of Error-Correcting Codes, North-Holland (1977).
[SB70] N. J. A. Sloane and E. R. Berlekamp, Weight enumerator for second-order Reed–Muller codes, IEEE Trans. Inf. Theory 16, 745 (1970).
[KT70] T. Kasami and N. Tokura, On the weight structure of Reed–Muller codes, IEEE Trans. Inf. Theory 16, 752 (1970).
[Car21] C. Carlet, Boolean Functions for Cryptography and Coding Theory, Cambridge University Press (2021), Sec. 4.1.5 (Ax's theorem).

B1

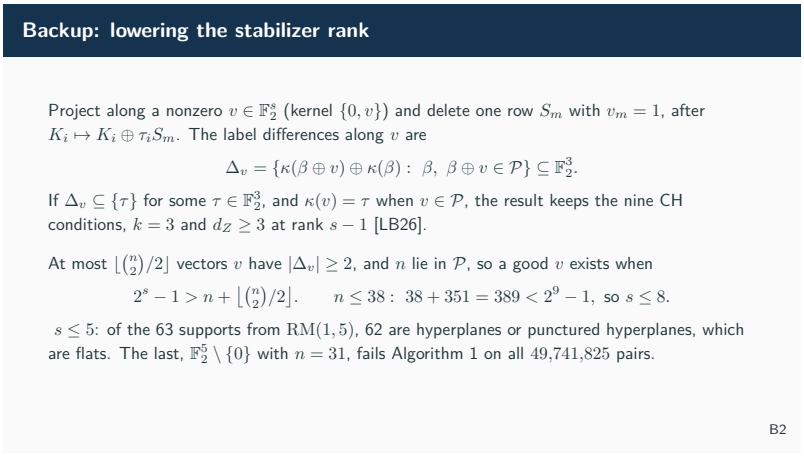
Clock questions only · **Clicks** 1

On screen: the admissible-length theorem and a table of Reed–Muller weights for $s = 5, 6, 7$. The click adds the affine-flat argument for $n = 15, 16$.

Use this if asked why the bound reduces to Reed–Muller weights. The conditions S1 to S3 hold exactly when χ_S, possibly after adding δ_0, is a codeword c of $\mathrm{RM}(s-4, s)$, and the length is $n = |c| - c(0)$. So every admissible length comes from a Reed–Muller weight. At ranks 5, 6 and 7 the weights up to 39 are known: from first-order codes, from the Sloane–Berlekamp enumerator, and from Kasami–Tokura together with Ax's theorem. That leaves the candidate lengths in the table.

[click] The two smallest candidates, 15 and 16, come from the minimum weight 16, whose support is an affine 4-flat. On such a flat the single-row conditions force each label κ_i to have low degree, so their product has degree below the dimension, and the triple overlap is even. That contradicts K3.

Backup B2. Backup: lowering the stabilizer rank



Clock questions only · **Clicks** 2

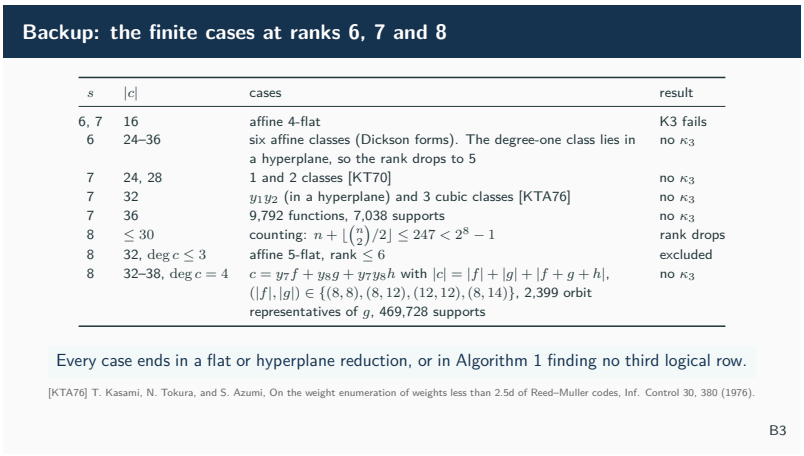
On screen: the projection along v, the label differences Δ_v, and the condition that keeps the CH conditions at rank $s - 1$. The clicks add the counting bound, then the case $s \leq 5$.

Use this if asked why only ranks 6, 7 and 8 need a search. Project the columns along a nonzero vector v and delete one stabilizer row. Columns that differ by v merge. If all such pairs have the same label difference τ, the merged columns carry equal labels, and the smaller code keeps the nine CH conditions, three logical qubits and $d_Z \geq 3$.

[click] Each pair of support points determines one difference v. So at most $\lfloor \binom{n}{2}/2 \rfloor$ vectors have two or more label differences, and n vectors lie in the support. If $2^s - 1$ exceeds their sum, a good v exists. For $n \leq 38$ that holds whenever $s \geq 9$, so $s \leq 8$.

[click] At rank 5 or less, every support but one is a flat, and the last one, all 31 nonzero points of $\mathbb{F}_2^5$, fails Algorithm 1 on every pair.

Backup B3. Backup: the finite cases at ranks 6, 7 and 8



Backup: the finite cases at ranks 6, 7 and 8

s	$\lvert c\rvert$	cases	result
6, 7	16	affine 4-flat	K3 fails
6	24–36	six affine classes (Dickson forms). The degree-one class lies in a hyperplane, so the rank drops to 5	no κ_3
7	24, 28	1 and 2 classes [KT70]	no κ_3
7	32	$y_1 y_2$ (in a hyperplane) and 3 cubic classes [KTA76]	no κ_3
7	36	9,792 functions, 7,038 supports	no κ_3
8	≤ 30	counting: $n + \lfloor \binom{n}{2}/2 \rfloor \leq 247 < 2^8 - 1$	rank drops
8	32, $\deg c \leq 3$	affine 5-flat, rank ≤ 6	excluded
8	32–38, $\deg c = 4$	$c = y_7 f + y_8 g + y_7 y_8 h$ with $\lvert c\rvert = \lvert f\rvert + \lvert g\rvert + \lvert f + g + h\rvert$, $(\lvert f\rvert, \lvert g\rvert) \in \{(8,8), (8,12), (12,12), (8,14)\}$, 2,399 orbit representatives of g, 469,728 supports	no κ_3

Every case ends in a flat or hyperplane reduction, or in Algorithm 1 finding no third logical row.

[KTA76] T. Kasami, N. Tokura, and S. Azumi, On the weight enumeration of weights less than 2.5d of Reed–Muller codes, Inf. Control 30, 380 (1976).

B3

Clock questions only · **Clicks** 0

On screen: a table of the cases at ranks 6, 7 and 8 and how each is excluded.

Use this if asked what the finite search covers. At rank 6, the Dickson forms leave six classes, and none admits a third logical row. At rank 7, the weights 24, 28 and 32 leave a few classes from Kasami–Tokura and Kasami–Tokura–Azumi, and weight 36 is enumerated directly. At rank 8, small weights reduce the rank by counting, and degree three forces an affine flat. The remaining degree-4 polynomials split as $y_7 f + y_8 g + y_7 y_8 h$, and the weight bound leaves four weight pairs for f and g. We list orbit representatives of g, enumerate h, and run Algorithm 1 on every resulting support. Every case ends in a flat or hyperplane reduction, or in no third logical row.

Backup B4. Backup: dictionary with [CLM+26]

Backup: dictionary with [CLM+26]

[CLM+26]	This talk
$U(b) = \bigotimes_j \mathrm{diag}(1, \omega^{b_j})$, $\omega = e^{2\pi i/N}$, $N = 2^\ell$	$N = 8$, $b \equiv \Gamma$: $T = U(1)$, $T^\dagger = U(-1)$
$b \cdot w$ for $w \in \mathbb{F}_2^n$	$\mathrm{sw}_\Gamma(w)$
$U(b)$ fixes the code iff	rows with a stabilizer row: $\mathrm{sw}_\Gamma(S_l) \equiv 0$ (8),
$b \in H_N = \left(\beta_2 \star \bigcup_{i<\ell} 2^i \beta_1^{(i)}\right)^{\perp_N}$	$\mathrm{sw}_\Gamma(S_l \wedge g) \equiv 0$ (4), $\mathrm{sw}_\Gamma(S_l \wedge g \wedge g') \equiv 0$ (2)
logical action $\omega^{b\cdot(Bv)} = \prod_J C_J\text{-}U(a_J)$,	CCZ $= C_{123}$-$U(4)$: $\mathrm{sw}_\Gamma(K_i) \equiv 0$ (8),
$a_J = (-2)^{\lvert J\rvert-1}\, b \cdot (w_{j_1} \star \cdots \star w_{j_h})$	$\mathrm{sw}_\Gamma(K_i \wedge K_{i'}) \equiv 0$ (4),
	$\mathrm{sw}_\Gamma(K_1 \wedge K_2 \wedge K_3) \equiv 1$ (2)

Our signed conditions are their H_8 with $b \in \{\pm 1\}^n$ and the logical action fixed to CCZ. We solve the converse problem: given the action, find S, K and b.

B4

Clock questions only · **Clicks** 0

On screen: a two-column table matching the notation of Camps-Moreno et al. 2026 with the signed conditions of this talk.

Use this if asked how this relates to the transversal-gates framework of Camps-Moreno, López, Matthews, Rengaswamy and San José. They write a diagonal transversal gate as $U(b)$ with phases ω^{b_j}, ω a 2^ℓ-th root of unity. Our $T/T^\dagger$ pattern is the case $N = 8$ with $b \equiv \Gamma$, and their $b \cdot w$ is our signed weight. Their group H_N of gates that fix the code gives our conditions on overlaps that include a stabilizer row. Their expansion of the logical action into multi-controlled phases gives our conditions on the logical rows, once we ask for exactly CCZ. So our signed conditions are their H_8 with $b \in \{\pm 1\}^n$ and the logical action fixed to CCZ. Our work is the converse problem: given the action, find S, K and b.

Background (not spoken)

- **Their notation.** Bases $\beta_2 \subseteq \beta_1$ of $\mathcal{C}_2 \subseteq \mathcal{C}_1$ are our rows of S and of G, and the logical basis vectors w_j are our rows K_j. $A^{(r)}$ is the set of products of r distinct elements of A.
- **Coefficient.** Their proposition on controlled phases writes a_J with $\mathrm{wt}(b \star w_{j_1} \star \cdots)$, where wt is the Hamming weight. That equals $b \cdot (w_{j_1} \star \cdots)$ when b is a 0/1 vector. Their lemma on the logical action, $\omega^{b\cdot(Bv)}$, gives the dot product in general, which is the reading needed when $T^\dagger$ entries are 7.
- **Their "transversal logical".** They call $U(b)$ a transversal logical when its logical action is a tensor product of single-qubit gates. Logical CCZ is not of that form, so our gate lies in H_8 but not in their T_8.

After the talk

Leave slide 28 on screen during questions. Backup slides B1 to B4 follow it: Reed–Muller lengths, rank reduction, the finite cases, and the dictionary with Camps-Moreno et al. Prepared answers to likely technical questions are in questions.md.